

LAB GUIDE WITH SOLUTIONS

Lab Guide

Atoms

Ten labs with full solutions · approx. 6 hours

Elixir Training Programme

Elixir · atoms

Session 5 · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

How to use this guide

Ten labs, in order, roughly six hours including discussion. Each lab states a goal, gives you the tasks, and then gives the full solution with the real output it produced on the training machine.

Atoms are the smallest idea in the language and the one that most often takes a production node down. They cost nothing to use and everything to abuse, and the line between the two is exactly one function call —

`String.to_atom/1`. Labs 1 to 8 build the intuition. Lab 9 shows you the failure. Lab 10 makes you write the code that cannot have it.

#	Lab	Minutes	Covers
1	What an atom is, and interning	30	<code>is_atom</code> , <code>:erts_debug.same/2</code> , <code>atom_count</code>
2	How an atom is written	25	quoting rules, <code>true/false/nil</code> , operators
3	Module names are atoms	30	<code>Elixir.prefix</code> , <code>apply/3</code> , <code>Module.concat</code>
4	What an atom costs	35	<code>flat_size</code> , the table, the GC, 255, <code>+t</code>
5	Ordering, and speed measured honestly	40	term order, three benchmarks, map keys
6	Atoms as tags	35	<code>ok/error</code> tuples, keyword lists, structs
7	A state machine of atoms	35	states, events, guards, <code>in</code>
8	Names outside your code	30	processes, ETS, <code>node()</code> , <code>config</code>
9	The denial of service	40	<code>to_atom</code> vs <code>to_existing_atom</code> , the JSON trap
10	Capstone: a safe event router	45	all of it, and a delta of zero

IMPORTANT Windows users: the launchers are `iex.bat` and `mix.bat`. In PowerShell, plain `iex` collides with PowerShell's own `Invoke-Expression` alias — always type `iex.bat`.

WATCH OUT Two starter files sit in `code/.atom_lab.exs` carries labs 1 and 4; `events.exs` is the capstone. Both run as handed out — they print `:todo` where your code goes.

READ THE DELTAS Every `atom_count` and `atom_used` in this guide is a count on THIS machine at THAT moment in the script. Yours will differ — a freshly booted VM here starts around 19,100 atoms. What must match is the DELTA: zero where the guide says zero, +10,000 where it says +10,000.

LAB 1 · 30 MINUTES · DEFINITION

What an atom is, and interning

GOAL Prove to yourself that two atoms with the same name are not two equal values — they are the same value, at the same address.

Task 1.1 — The basic checks

In IEx, establish what is and is not an atom, and how atoms compare.

Solution

```
iex> is_atom(:ok)
true
iex> is_atom(true)      # and nil, and false
true
iex> is_atom(String)
true
iex> is_atom("ok")
false
iex> :ok == :ok
true
iex> :ok === :ok
true
iex> :ok == "ok"
false
```

WHY An atom is a constant whose name IS its value. `==` and `===` cannot differ for two atoms, because there is no type to tell apart and no representation to convert. `:ok == "ok"` is false and always will be — the quotes make it a different kind of thing entirely.

Task 1.2 — Build the same atom three different ways

Make one atom as a literal, one with `String.to_atom/1`, and one by round-tripping through `:erlang.term_to_binary/1`. Then ask whether they are equal — and then ask the harder question, whether they are the same word in memory. `:erts_debug.same/2` answers that one; it is a debug function, so it is a lab tool, not production code.

Solution

```
a1 = :lab5_interned_name           # a literal
a2 = String.to_atom("lab5_interned_name") # built at run time
a3 = :erlang.binary_to_term(:erlang.term_to_binary(a1)) # round-tripped

a1 == a2 and a2 == a3              = true
:erts_debug.same(a1, a2)           = true    <- the SAME word, not a copy
:erts_debug.same(a1, a3)           = true
```

WHY Three completely different routes, one value. The compiler put the first in the table; the other two looked it up and found it already there. That is interning.

Task 1.3 — Now do it with binaries and watch it fail

Repeat the test with two equal strings. Predict the two answers before you run it.

Solution

```
b1 = "lab5_interned_name"
b2 = String.duplicate("lab5_interned_name", 1)

b1 == b2                = true
:erts_debug.same(b1, b2) = false <- equal, but two objects
```

WHY This is the whole difference. Two equal binaries are two blocks of memory that happen to hold the same bytes; comparing them may have to walk both. Two equal atoms are one word compared with itself. Lab 5 measures what that is worth.

Task 1.4 — Create the same atom a hundred thousand times

Take `:erlang.system_info(:atom_count)`, create the SAME atom 100,000 times, and take the count again. Predict the delta.

Solution

```
before = :erlang.system_info(:atom_count)
for _ <- 1..100_000, do: String.to_atom("lab5_interned_name")
after_c = :erlang.system_info(:atom_count)

atom_countbefore 100,000 more String.to_atom calls = 19121
atom_count after                                = 19121
delta                                                    = 0
```

WHY Zero. `String.to_atom/1` on a name that already exists is a table LOOKUP, not an insert. Everything dangerous about that function in lab 9 comes from the other case — a name the table has never seen.

CHECKPOINT You can explain, without using the word “pointer”, why `:ok == :ok` is cheaper than `"ok" == "ok"`, and you can name the one condition under which `String.to_atom/1` grows the table.

LAB 2 · 25 MINUTES · LITERALS

How an atom is written

GOAL Learn which names need quotes, which atoms you have been using without noticing, and what the standard library gives you for converting them.

Task 2.1 — Build the quoting table

Try each of these forms. For each one, note whether it needed quotes to be written, and whether `inspect/1` prints the quotes back.

Solution

written as	inspects as	byte_size
<code>:ok</code>	<code>:ok</code>	2
<code>:hello_world</code>	<code>:hello_world</code>	11
<code>:valid?</code>	<code>:valid?</code>	6
<code>:save!</code>	<code>:save!</code>	5
<code>:Upper</code>	<code>:Upper</code>	5
<code>:_leading</code>	<code>:_leading</code>	8
<code>:"hello world"</code>	<code>:"hello world"</code>	11
<code>:"with-dash"</code>	<code>:"with-dash"</code>	9
<code>:"123"</code>	<code>:"123"</code>	3
<code>:""</code>	<code>:""</code>	0
<code>:"user@host"</code>	<code>:user@host</code>	9
<code>:héllö</code>	<code>:héllö</code>	6

WHY Unquoted, a name may start with a letter or `_` and continue with letters, digits and `_`, and may end with one `?` or `!`. Anything else needs quotes — a space, a dash, a leading digit, the empty name. Two of these are worth noticing: `:"user@host"` needs quotes to WRITE but inspects without them, because `@` is legal in the middle of a name; and `:héllö` is six bytes for five characters, because the name is stored as UTF-8.

Task 2.2 — Atoms you have been using since day one

Check what `true`, `false` and `nil` actually are.

Solution

```

true  == :true  = true
false == :false = true
nil   == :nil   = true
inspect(:true) = true      inspect(:nil) = nil
is_boolean(true) = true    is_boolean(:maybe) = false

```

WHY There is no boolean type. `true` is the atom `:true` with nicer printing, and `is_boolean/1` only answers true for those two particular atoms. `nil` is an atom too — which is why a missing value and a present-but-nil value look identical, and why `Map.fetch/2` exists.

Task 2.3 — Operators are atoms as well

Check `:+` and `:<>`, then pull the first element out of a quoted expression.

Solution

```

is_atom(:+) = true    inspect(:+) = :+
is_atom(:<>) = true    inspect(:<>) = :<>

```

```
elem(quote(do: 1 + 2), 0) = :+
```

WHY Session 4 said an operator is a function with a symbol for a name. This is the other half of that sentence: the name is an atom, and it sits in the first slot of every node in the AST.

Task 2.4 — The conversion functions, and how small the module is

Solution

```
Atom.to_string(:ok)      = "ok"
Atom.to_charlist(:ok)    = ~c"ok"
String.to_atom("ok")     = :ok

Atom.__info__(:functions) = [to_char_list: 1, to_charlist: 1, to_string: 1]
```

WHY Three functions, one of them a deprecated spelling of another. There is nothing to learn about the `Atom` module, because an atom has no structure to operate on. The interesting functions all live in `String` and `Kernel`, and they go the other way.

CHECKPOINT Given a name, you can say immediately whether it needs quotes, and you can explain why `true` is not a keyword.

LAB 3 · 30 MINUTES · MODULES

Module names are atoms

GOAL See through the capital letter: every module reference you write is an atom with a prefix, and that is what makes dynamic dispatch possible.

Task 3.1 — Unmask a module name

Compare `String` with the atom `:Elixir.String`, then look at what the name really is.

Solution

```
String == :Elixir.String      = true
is_atom(String)              = true
Atom.to_string(String)       = "Elixir.String"
Atom.to_string(Enum)         = "Elixir.Enum"
Atom.to_string(:erlang)      = "erlang"
inspect(:Elixir.String)      = String
```

WHY A capitalised name in source is sugar: the compiler prefixes it with `Elixir.` and hands the atom on. Erlang modules have no prefix, which is why you write `:erlang` with a colon and `Enum` without one — they are exactly the same kind of value underneath.

Task 3.2 — Build a module name

Use `Module.concat/2` and check what it produced.

Solution

```
Module.concat(Foo, Bar)      = Foo.Bar
Atom.to_string(Module.concat(Foo, Bar)) = "Elixir.Foo.Bar"
```

WHY A nested module name is one flat atom with dots in it. `Foo.Bar` is not “inside” `Foo` in any structural sense — it is a separate module whose name happens to share a prefix.

Task 3.3 — Dispatch on values

Call a function three ways: statically, through `apply/3`, and with both the module and the function name held in variables.

Solution

```
apply(String, :upcase, ["hi"]) = "HI"
apply(:erlang, :length, [[1,2,3]]) = 3

m = String.to_existing_atom("Elixir.String")
f = :downcase
apply(m, f, ["LOUD"]) = "loud"
function_exported?(String, :upcase, 1) = true
```

WHY This is what atoms buy you. Because a module is a value, the call target can be decided at run time — which is how plugins, adapters and behaviours work. Note `to_existing_atom`, not `to_atom`: if the module name came from outside your code, lab 9 is about what `to_atom` would do to you.

Task 3.4 — An atom names nothing in particular

Make an atom for a module that does not exist, and try to call it.

Solution

```
is_atom(:"Elixir.NoSuchModule")    = true
Code.ensure_loaded?(NoSuchModule)  = false
```

```
NoSuchModule.run()
```

```
** (UndefinedFunctionError) function NoSuchModule.run/0 is undefined
   (module NoSuchModule is not available). Make sure the module name is
   correct and has been specified in full (or that an alias has been defined)
```

WHY The atom exists the moment you write it. Whether any code answers to that name is a completely separate question, answered only when you call it. An atom is a NAME, not a reference.

CHECKPOINT You can say what Enum evaluates to, and why NoSuchModule is not a compile error.

LAB 4 · 35 MINUTES · COST

What an atom costs

GOAL Measure both halves of the price: free inside a term, permanent in the table.

Use `code/atom_lab.exs` for this lab, or work straight in IEx.

Task 4.1 — The cheap half

Measure `:erts_debug.flat_size/1` for atoms of very different lengths, then for a tuple holding atoms and a tuple holding a string.

Solution

```
:ok                                flat_size = 0 words
:error                            flat_size = 0 words
:a_much_longer_atom_name_than_that_one flat_size = 0 words
{:ok, :error}                    flat_size = 3 words
{:ok, "error"}                   flat_size = 6 words
word size = 8 bytes
```

WHY Zero words, whatever the name. An atom is an immediate — it fits inside the word that would otherwise hold a pointer, so it costs nothing on the process heap and nothing to copy between processes. The tuple costs 3 words either way: a header and two slots. Putting a string in one of those slots adds 3 more words for the binary itself.

Task 4.2 — The expensive half

Look at the table: how many atoms exist, how many are allowed, and how much memory they occupy.

Solution

```
atom_count = 19234
atom_limit = 1048576
memory(:atom_used) = 577896 bytes
headroom    = 1029041 atoms
```

WHY A bare VM already holds about nineteen thousand atoms — every module name, every function name, every option in the standard library. The limit is a little over a million by default, and it is a hard ceiling on the whole NODE, not on your process.

Task 4.3 — Prove they are never collected

Create 10,000 atoms you immediately throw away, force a full garbage collection with `:erlang.garbage_collect/0`, and measure again.

Solution

```
bc = :erlang.system_info(:atom_count)
bm = :erlang.memory(:atom_used)
for i <- 1..10_000, do: String.to_atom("lab5_throwaway_atom_number_#{i}")
:erlang.garbage_collect()

before 10,000 throwaway atoms:    count=19535  atom_used=577948 bytes
after them, and after a full GC:  count=29535  atom_used=1091642 bytes
delta: count=+10000  atom_used=+513694 bytes (51.4 bytes each)
```

WHY Half a megabyte, gone for the lifetime of the node, for 10,000 values nothing refers to any more. The garbage collector does not touch the atom table — it cannot, because there is no way to know that no future code will name that atom again. This is the single fact that makes lab 9 possible.

Task 4.4 — The other hard limit

Find the longest atom this VM will accept. Start at 255 characters.

Solution

```
255 characters -> no error (byte_size 255)
256 characters -> ** (SystemLimitError) a system limit has been reached
```

WHY 255 characters, not bytes — a name of 255 accented characters is longer than 255 bytes and still legal. `SystemLimitError` is not something you can rescue and carry on from in any meaningful way; treat it as a crash.

Task 4.5 — Move the ceiling

The limit can be raised — but only at boot. Launch a child VM with and without the `+t` flag and ask each one for its limit.

Solution

```
# a file containing just: IO.puts(:erlang.system_info(:atom_limit))

elixir child.exs -> 1048576
elixir --erl "+t 2000000" child.exs -> 2000000
```

WHY There is no runtime call that changes this: the table is sized when the node boots. Raising `+t` buys headroom, it does not fix a leak — a program that turns input into atoms will reach two million just as reliably as it reached one.

CHECKPOINT You can answer both halves of “what does an atom cost?” — zero words in the term, about fifty bytes forever in the table — and you know which half matters.

LAB 5 · 40 MINUTES · MEASUREMENT

Ordering, and speed measured honestly

GOAL Sort atoms, place them in the term order, and then run the benchmark that everyone quotes — correctly.

Task 5.1 — Sort by name, not by age

Create `:Zulu` last, then sort a list that contains it, and see where it lands.

Solution

```
Enum.sort([:zebra, :apple, :mango, :Zulu]) = [:Zulu, :apple, :mango, :zebra]
:error < :ok      = true
:Z < :a           = true      <- uppercase first: codepoint order
```

WHY Atoms order by their NAME, byte by byte in UTF-8, not by the order they were created. That makes sorted output deterministic across runs and across nodes — and it means `:Zulu` sorts before `:apple`, which surprises people who expect case-insensitive order.

Task 5.2 — Place atoms in the term order

Compare an atom with a number, a tuple and a binary. Then sort a list of six different types.

Solution

```
1 < :ok      = true
:ok < {1}     = true
:ok < "ok"    = true

Enum.sort([:b, 2, "a", [1], {1}, %{}]) = [2, :b, {1}, %[], [1], "a"]
```

WHY `number < atom < reference < function < port < pid < tuple < map < list < bitstring`. Atoms sit second, just above numbers. Nothing raises — which is exactly the guard trap from session 4: `"ok" > :ok` is true, so a guard that compares before it type-checks will accept an atom.

Task 5.3 — The benchmark, done three ways

Time a million comparisons of an atom against itself, then a million comparisons of a binary. Do the binary case TWICE: once against the same binary object, and once against a million equal but separately built binaries. Predict all three.

Solution

```
n = 1_000_000
atoms      = List.duplicate(:some_moderately_long_atom_name, n)
same_bin   = List.duplicate("some_moderately_long_atom_name", n)
distinct   = for _ <- 1..n, do: String.duplicate("some_moderately_long_atom_name", 1)

atom == atom                : 10547 us
binary == the SAME binary object : 17203 us (ratio 1.63x)
binary == an equal but DISTINCT binary: 29081 us (ratio 2.76x)
```

WHY The middle row is the one that lies, and it is the one most blog posts measure. `List.duplicate/2` stores ONE binary a million times, so `==` compares a pointer with itself and never looks at a byte. Parsed input does not do that — every request builds its own binary — so the third row is the one that describes your code. It is also the least stable row from run to run: on the machine that built deck 5 the middle row came out FASTER than the atom. Run it twice before you quote it.

Task 5.4 — Where it actually shows up: map keys

Time a million lookups in a one-key map, atom key against string key. Then measure what each map costs.

Solution

```
1000000 map lookups: atom key 19456 us    string key 46387 us    ratio 2.38x

a 1-key atom map    flat_size = 6 words
a 1-key string map  flat_size = 9 words
a 3-key atom map    flat_size = 10 words
a 3-key string map  flat_size = 19 words
```

WHY The key has to be HASHED before anything is compared, and hashing an atom is hashing one word. So the atom key wins twice — faster to look up, and smaller to store, because the key's bytes live in the table instead of in every copy of the map. This is the practical reason internal maps use atom keys.

CHECKPOINT Someone shows you a benchmark proving strings are as fast as atoms. You can tell in ten seconds whether it built its binaries with `List.duplicate/2`, and you can say why that matters.

LAB 6 · 35 MINUTES · IDIOM

Atoms as tags

GOAL Write the three idioms that account for most of the atoms in real Elixir: the tagged tuple, the keyword list and the struct.

Task 6.1 — The ok/error tuple

Write `Accounts.fetch/1` returning `{:ok, user}` or `{:error, reason}`, with a separate error for an id that is not an integer at all. Then write `Accounts.describe/1`, which dispatches on the tag.

Solution

```
defmodule Accounts do
  @users %{1 => %{name: "Ada", role: :admin},
          2 => %{name: "Ravi", role: :viewer}}

  def fetch(id) when is_integer(id) do
    case Map.fetch(@users, id) do
      {:ok, user} -> {:ok, user}
      :error      -> {:error, :not_found}
    end
  end

  def fetch(_), do: {:error, :bad_id}

  def describe(id) do
    case fetch(id) do
      {:ok, %{name: n, role: :admin}} -> "#{n} can do anything"
      {:ok, %{name: n, role: :viewer}} -> "#{n} is a #{r}"
      {:error, :not_found}             -> "no such user"
      {:error, :bad_id}                -> "that is not an id"
    end
  end
end
```

```
fetch(1) = {:ok, %{name: "Ada", role: :admin}} -> Ada can do anything
fetch(2) = {:ok, %{name: "Ravi", role: :viewer}} -> Ravi is a viewer
fetch(99) = {:error, :not_found} -> no such user
fetch("1") = {:error, :bad_id} -> that is not an id

File.read("no_such_file") = {:error, :enoent}
Map.fetch(%{a: 1}, :b) = :error
```

WHY The tag is the point. The atom in slot one says which shape slot two has, so a single case can both branch and destructure. `File.read/1` uses `:enoent` — the same atom the C library used for “no such file” — and `Map.fetch/2` answers with the bare atom `:error` rather than a tuple, because there is no reason to carry one.

Task 6.2 — Options, via keyword lists

Write `Report.run/1` taking options with defaults for `:sort`, `:limit` and `:format`. Then prove to yourself that the `[key: value]` syntax is nothing but a list of two-element tuples.

Solution

```
defmodule Report do
  @defaults [sort: :asc, limit: 10, format: :text]

  def run(opts \\ []) do
    opts = Keyword.merge(@defaults, opts)
    {Keyword.fetch!(opts, :sort), Keyword.fetch!(opts, :limit),
     Keyword.fetch!(opts, :format)}
  end
end

[sort: :asc, limit: 10] == [{:sort, :asc}, {:limit, 10}] -> true
Keyword.keyword?([sort: :asc])      = true
Keyword.keyword?(["sort", :asc])     = false

Report.run()                        = {:asc, 10, :text}
Report.run(limit: 3)                = {:asc, 3, :text}
Report.run(sort: :desc, format: :csv) = {:desc, 10, :csv}
```

WHY A keyword list is a list of {key, value} pairs where every key is an atom — `Keyword.keyword?/1` says so, and refuses a list with a string key. The `[key: value]` sugar, the trailing-options convention and the whole `Keyword` module exist only because the key is an atom.

Task 6.3 — Structs, where the keys are atoms by construction

Define a `User` struct with a `:role` field defaulting to an atom, build one, and then try to read it with a string key.

Solution

```
defmodule User do
  defstruct name: "", role: :viewer, active: true
end

u = %User{name: "Ada", role: :admin}

u
  = %User{name: "Ada", role: :admin, active: true}
u.__struct__
  = User    is_atom = true
Map.keys(Map.from_struct(u))
  = [:active, :name, :role]
u.role == :admin
  = true

Map.fetch!(u, "name")
** (KeyError) key "name" not found in:
```

WHY A struct is a map whose keys are atoms and which carries one more: `__struct__`, holding the module's own name. That is how pattern matching on `%User{}` works at all. The string key is not a near miss — it is a different key, and on Elixir 1.20 the type checker will warn at compile time that this call can only ever raise.

CHECKPOINT You can write a function that returns `{:ok, value}` or `{:error, reason}` without thinking about it, and you can say why a keyword list cannot have string keys.

LAB 7 · 35 MINUTES · IDIOM

A state machine of atoms

GOAL Build something whose entire vocabulary is atoms — states, events and failure reasons — and see how much of the correctness the compiler and the guards give you for free.

Task 7.1 — States and transitions

An order moves through `:cart`, `:paid`, `:shipped` and `:delivered`, and can be `:cancelled` from the first two. Write `Order.next/2` returning `{:ok, new_state}` or `{:error, reason}`.

Solution

```
defmodule Order do
  @states [:cart, :paid, :shipped, :delivered, :cancelled]
  @table %{
    cart:      [pay: :paid, cancel: :cancelled],
    paid:      [ship: :shipped, refund: :cancelled],
    shipped:    [deliver: :delivered],
    delivered: [],
    cancelled: []
  }

  def states, do: @states

  def next(state, event) when state in @states and is_atom(event) do
    case Keyword.fetch(@table[state], event) do
      {:ok, new_state} -> {:ok, new_state}
      :error            -> {:error, {:illegal, state, event}}
    end
  end

  def next(state, _event), do: {:error, {:unknown_state, state}}
end
```

WHY Every name here is an atom: the states, the events, the two failure reasons. `state in @states` compiles to a chain of `==` on atoms — each one a single word comparison — so the guard costs almost nothing. And because the transition table is a module attribute, it is built once at compile time, not on every call.

Task 7.2 — Exercise every branch

Solution

```
states = [:cart, :paid, :shipped, :delivered, :cancelled]

next(:cart, :pay)      = {:ok, :paid}
next(:paid, :ship)     = {:ok, :shipped}
next(:shipped, :deliver) = {:ok, :delivered}
next(:delivered, :cancel) = {:error, {:illegal, :delivered, :cancel}}
next(:cart, :ship)     = {:error, {:illegal, :cart, :ship}}
next(:frozen, :pay)    = {:error, {:unknown_state, :frozen}}
```

WHY The two failures are told apart by their tag, not by a string. `{:illegal, state, event}` is something a caller can match on and log; `{:unknown_state, s}` means the caller has a bug, and deserves a different reason.

Task 7.3 — Fold a whole sequence

Write `Order.run/1`, which starts at `:cart` and applies a list of events, stopping at the first failure and keeping its reason.

Solution

```
def run(events), do: Enum.reduce(events, {:ok, :cart}, &step/2)

defp step(event, {:ok, state}), do: next(state, event)
defp step(_event, {:error, _} = e), do: e

run([:pay, :ship, :deliver]) = {:ok, :delivered}
run([:pay, :deliver])       = {:error, {:illegal, :paid, :deliver}}
run([:cancel, :pay])        = {:error, {:illegal, :cancelled, :pay}}
```

WHY Two clauses, matched on the tag, and the error carries itself to the end untouched. This is the tagged tuple earning its keep: no flags, no accumulator struct, no early return.

Task 7.4 — Now break it on purpose

Add `:refund` to the transition table for `:shipped`, but spell it `:refnud` in the table and `:refund` in your test call. What does the machine report, and at what point would you have found out?

WATCH OUT You get `{:error, {:illegal, :shipped, :refund}}` — a perfectly ordinary “no such transition”. Nothing warns you: `:refnud` is a valid atom, so the typo is a legal program that is simply wrong. That is the cost of atoms as a vocabulary, and the reason the list of legal states and events belongs in ONE module attribute that everything else reads.

CHECKPOINT Your machine rejects every illegal transition, tells an illegal event apart from an unknown state, and folds a list of events without a single `if`.

LAB 8 · 30 MINUTES · RUNTIME

Names outside your code

GOAL Meet the atoms that name things the VM owns: processes, tables, nodes and configuration keys.

Task 8.1 — Name a process and send to the name

Spawn a process that loops, register it under an atom, find it by that atom, and send it a message without ever holding its pid.

Solution

```
defmodule Worker do
  def loop do
    receive do
      {:ping, from} ->
        send(from, {:pong, self()})
        loop()
    end
  end
end

pid = spawn(&Worker.loop/0)
Process.register(pid, :lab5_worker)
send(:lab5_worker, {:ping, self()})

Process.whereis(:lab5_worker) = #PID<0.117.0>
that pid is                  = #PID<0.117.0>
send by NAME reached #PID<0.117.0> -> got {:pong, ...}
```

WHY A registered name is a node-wide atom, so any process can reach the worker without being handed its pid. It must LOOP — a process that has already exited releases its name, and the next task will not fail the way it should.

Task 8.2 — Take a name that is already taken

Solution

```
Process.register(self(), :lab5_worker)
** (ArgumentError) could not register #PID<0.94.0> with name :lab5_worker
    because it is not alive, the name is already taken, or it has already
    been given another name

length(Process.registered()) = 33
```

WHY One atom, one process, node-wide. Thirty-three names are already registered on a bare node — `:user`, `:kernel_sup`, `:logger` and the rest of the kernel. This is the namespace GenServer writes into when you pass it `name: :my_server`.

Task 8.3 — Tables, nodes and configuration

Create a named ETS table, put something in it, and read it back. Then look at `node()` and at two configuration lookups.

Solution

```
:ets.new(:lab5_table, [:set, :named_table]) -> :lab5_table
:ets.insert(:lab5_table, {:region, "apac"})
:ets.lookup(:lab5_table, :region)           -> [region: "apac"]

node()                                     = :nonode@nohost
Application.spec(:elixir, :vsns)           = ~c"1.20.4"
Application.get_env(:elixir, :ansi_enabled) = false
Application.get_env(:elixir, :nope, :default) = :default
```

WHY Count the atoms in those five lines: the table name, both table OPTIONS, the key, the node name, the application name and both config keys. `[region: "apac"]` is a keyword list, which is why it prints with the key: sugar. Almost every name the runtime owns is an atom, and all of them were written in someone's source code — not derived from input.

CHECKPOINT You can name four things outside your own code that an atom identifies, and you can say what happens when two processes want the same registered name.

LAB 9 · 40 MINUTES · SAFETY

The denial of service

GOAL Do the damage deliberately, on a machine you can restart, so that you recognise it in code review afterwards.

Task 9.1 — Simulate a handler that atomises its input

Write the handler exactly as it appears in real code, then push 5,000 “requests” through it and measure.

Solution

```
defmodule Unsafe do
  def handle(body), do: String.to_atom(body)
end

start_c = :erlang.system_info(:atom_count)
for i <- 1..5_000, do: Unsafe.handle("lab5_request_tag_#{i}")

atom_count 31277 -> 36277 (+5000, permanent)
headroom left: 1012299
at this rate the node aborts after ~202 more batches
```

WHY Five thousand requests, five thousand atoms, none of which will ever be collected. Two hundred more batches of the same size and the table is full. When it fills, the VM does not raise something you can rescue — it ABORTS the node, taking every process, every connection and every supervisor with it. One unauthenticated endpoint is enough.

Task 9.2 — The safe door

Replace `String.to_atom/1` with `String.to_existing_atom/1`. Call it once with a name that exists and once with a name that does not, measuring the count either side of the failure.

Solution

```
String.to_existing_atom("ok") = :ok

String.to_existing_atom("lab5_never_seen_before_xyz")
** (ArgumentError) errors were found at the given arguments:

atom_count around that failed call: 36277 -> 36277 (delta 0)
```

WHY It raises `INSTEAD` of growing the table, and that is the entire point. An `ArgumentError` you can rescue and turn into a 400 is a bug report; a full atom table is an outage.

Task 9.3 — The JSON trap

You have decoded a payload into a map with string keys and you want atom keys. Write three versions — the unsafe one, one built on `to_existing_atom`, and one that whitelists the keys you accept — and run them against a payload with one key you never asked for.

Solution

```
defmodule Keys do
  @allowed [ :name, :role ]

  def unsafe(map), do: Map.new(map, fn {k, v} -> {String.to_atom(k), v} end)
```

```
def existing(map) do
  Map.new(map, fn {k, v} ->
    try do
      {String.to_existing_atom(k), v}
    rescue
      ArgumentError -> {k, v}
    end
  end)
end

def whitelist(map) do
  @allowed
  |> Map.new(fn k -> {k, Map.get(map, Atom.to_string(k))} end)
  |> Map.reject(fn {_k, v} -> is_nil(v) end)
end
```

```
payload = %{"evil_key_12345" => "x", "name" => "Ada", "role" => "admin"}

existing -> %{:name => "Ada", :role => "admin", "evil_key_12345" => "x"}
whitelist -> %{name: "Ada", role: "admin"}
unsafe -> %{name: "Ada", role: "admin", evil_key_12345: "x"}

...and now run `existing` a SECOND time:
existing -> %{name: "Ada", role: "admin", evil_key_12345: "x"}
```

THE REAL LESSON Read the last two lines again. `existing/1` was safe the first time and unsafe the second, because `unsafe/1` had created that atom in between. `to_existing_atom` is not a property of the function — it is a property of the atom table at that instant, and anything else in the node can change it. Only the `whitelist` is safe no matter what else has run. Note also that `existing/1` returns a map with MIXED key types, which every downstream pattern match has to cope with.

Task 9.4 — Find the call in real code

Search a codebase you have — your own, or any dependency in `deps/` — for `String.to_atom`, `List.to_atom` and `:erlang.binary_to_atom`. For each hit, answer one question: can the argument come from outside the program?

WATCH OUT The rule that survives code review: `String.to_atom/1` is legal on a literal or on a value you constructed, and never on anything that crossed a network, a file or a form field. If you cannot point at the line where the value was born, use `to_existing_atom` and rescue it.

CHECKPOINT You can state the failure mode in one sentence — the node aborts, it does not raise — and you can explain why `String.to_existing_atom/1` is a weaker guarantee than it looks.

LAB 10 · 45 MINUTES · ALL OF IT

Capstone: a safe event router

GOAL Put the whole session into one module: parse hostile input into atom-keyed structs, dispatch on the tag, and finish with an atom-count delta of exactly zero.

Start from code/events.exs. The raw events are given; Event.parse/1, Router.handle/1, Router.run/1 and Router.summary/1 are yours.

Task 10.1 — Parse, with a whitelist

Event.parse/1 takes a map with string keys and returns {:ok, %Event{}} or one of five tagged errors. Every atom it can ever produce must be written in your source.

Solution

```
defmodule Event do
  defstruct [:type, :user, :amount]

  @types %{"signup" => :signup, "purchase" => :purchase, "refund" => :refund}

  def parse(%{} = raw) do
    with {:ok, type} <- fetch_type(raw),
         {:ok, user} <- fetch_user(raw) do
      {:ok, %Event{type: type, user: user, amount: Map.get(raw, "amount")}}
    end
  end

  def parse(_), do: {:error, :not_a_map}

  defp fetch_type(raw) do
    case Map.fetch(raw, "type") do
      {:ok, t} -> tag(Map.fetch(@types, t), :unknown_type)
      :error   -> {:error, :missing_type}
    end
  end

  defp fetch_user(raw) do
    case Map.fetch(raw, "user") do
      {:ok, u} when is_binary(u) -> {:ok, u}
      {:ok, _}                   -> {:error, :bad_user}
      :error                     -> {:error, :missing_user}
    end
  end

  defp tag({:ok, v}, _reason), do: {:ok, v}
  defp tag(:error, reason),   do: {:error, reason}
end
```

WHY @types is the whitelist, and it is a map rather than a list so the lookup is a lookup rather than a scan. An unknown type does not become an atom — it becomes {:error, :unknown_type}. The with has no else, because every step already returns a tagged tuple in exactly the shape the caller wants.

Task 10.2 — Dispatch on the struct

One `Router.handle/1` clause per event type, matching on the struct. A purchase whose amount is not a positive integer is `{:error, :bad_amount}`.

Solution

```
defmodule Router do
  def handle(%Event{type: :signup, user: u}), do: {:ok, "welcome #{u}"}

  def handle(%Event{type: :purchase, user: u, amount: a})
    when is_integer(a) and a > 0,
    do: {:ok, "#{u} paid #{a}"}

  def handle(%Event{type: :purchase}), do: {:error, :bad_amount}

  def handle(%Event{type: :refund, user: u, amount: a}),
    do: {:ok, "#{u} refunded #{a}"}

  def run(raws) do
    Enum.map(raws, fn raw ->
      case Event.parse(raw) do
        {:ok, event} -> handle(event)
        {:error, reason} -> {:error, reason}
      end
    end)
  end

  def summary(results) do
    Enum.frequencies_by(results, fn
      {:ok, _} -> :ok
      {:error, _} -> :error
    end)
  end
end
```

WHY Clause order matters: the guarded `:purchase` clause must come before the unguarded one, or every purchase would be `{:error, :bad_amount}`. This is session 3's rule meeting session 5's vocabulary.

Task 10.3 — Run the batch, and measure the damage

Take `:erlang.system_info(:atom_count)` either side of the batch. If your parser is right, the delta is zero — even though one of those events was written specifically to make you call `String.to_atom/1`.

Solution

```
%{"type" => "signup", "user" => "ada"} -> {:ok, "welcome ada"}
%{"amount" => 250, "type" => "purchase", ...} -> {:ok, "ravi paid 250"}
%{"amount" => 0, "type" => "purchase", ...} -> {:error, :bad_amount}
%{"amount" => 99, "type" => "refund", ...} -> {:ok, "kiran refunded 99"}
%{"type" => "drop_tables", "user" => "mallory"} -> {:error, :unknown_type}
%{"user" => "nobody"} -> {:error, :missing_type}
%{"type" => "signup", "user" => 42} -> {:error, :bad_user}
"not a map" -> {:error, :not_a_map}

summary = %{error: 5, ok: 3}
atom_count before the batch = 36403
atom_count after the batch = 36403 (delta 0)
```

WHY Zero. Every atom in the output — `:ok`, `:error`, `:unknown_type`, `:bad_user` — was written in your source and created at compile time. `"drop_tables"` produced a string comparison and a tagged error, and the table never moved.

Task 10.4 — Two things that should fail

Solution

```
Event.parse(%{"type" => "drop_tables", "user" => "mallory"})
#=> {:error, :unknown_type}

Router.handle(%{type: :signup})      # a plain map, not an %Event{}
** (FunctionClauseError) no function clause matching in Router.handle/1
```

WHY The second one is deliberate. `handle/1` matches on `%Event{}`, not on `%{}`, so a bare map — however right its keys look — never reaches a handler. The struct's `__struct__` field is doing the work.

Task 10.5 — Extend it (if there is time)

Pick one: add a `:chargeback` event whose amount must be negative; make `Router.summary/1` return `%{ok: n, error: n}` and a per-type breakdown; or add a `Router.audit/1` function that reports which raw types were rejected, WITHOUT turning any of them into atoms.

WATCH OUT The audit is the interesting one. You want to report the rejected type — a string the attacker chose — so it must stay a string all the way to the log line. The moment you reach for `String.to_atom/1` to “tidy it up”, you have rebuilt the vulnerability you spent lab 9 removing.

CHECKPOINT Your router refuses every hostile input with a tagged tuple, and running the batch a thousand times would still leave `atom_count` where it started.

Where this goes next

You now have the vocabulary that every later topic is written in. Tagged tuples, keyword options, struct keys, process names and message tags are all atoms, and GenServer, Supervisor and Ecto will assume you read them at a glance.

If you remember one thing per lab	It is this
1 · What an atom is	the same name is the same word — proved with <code>:erts_debug.same/2</code>
2 · How one is written	true, false and nil are atoms; so are <code>:+</code> and <code>:<></code>
3 · Module names	String is <code>:"Elixir.String"</code> , which is why <code>apply/3</code> works
4 · Cost	zero words in the term, ~50 bytes in the table, forever
5 · Speed	benchmark against DISTINCT binaries or you are measuring nothing
6 · Tags	the atom in slot one says what shape slot two has
7 · State machines	the whole vocabulary in one attribute, so a typo has one home
8 · Runtime names	processes, ETS tables, nodes and config keys are all atoms
9 · The danger	a full atom table aborts the node; it does not raise
10 · Capstone	whitelist at the boundary, and the delta stays zero

The one-line rule

Atoms are for names YOU write. Strings are for data OTHER PEOPLE send. Every bug in lab 9 is that sentence being broken in one direction.

MEASURED Every output in this guide was produced by `../verify/probe_labs5.exs` on Elixir 1.20.4 / Erlang/OTP 29, erts 17.0.6, 8-byte word, 12 schedulers. Timings and absolute atom counts will differ on your machine; the deltas, the values and the error messages will not. If yours disagree, your machine is right — say so and work out why.